HARNESS LAB · MÓDULO 2 · AULA 2.1

Hooks: o que o modelo *não decide*

Um programa seu, que o Claude Code roda em momentos certos da sessão, com o modelo concordando ou não. Pedir não é obrigar: o que está no CLAUDE.md é pedido, o que está num hook acontece.

`settings.json` `evento` `matcher` `SessionStart` `PreToolUse` `PostToolUse` `Notification` `exit 0 · exit 2` `allow · ask · deny`

VOCÊ SAI COM

Um hook espião que mostra o ciclo de vida da sua sessão evento por evento, um `SessionStart` que conta ao modelo a branch e a próxima tarefa, um aviso quando ele para esperando você, e o contrato do hook testado no seu terminal.

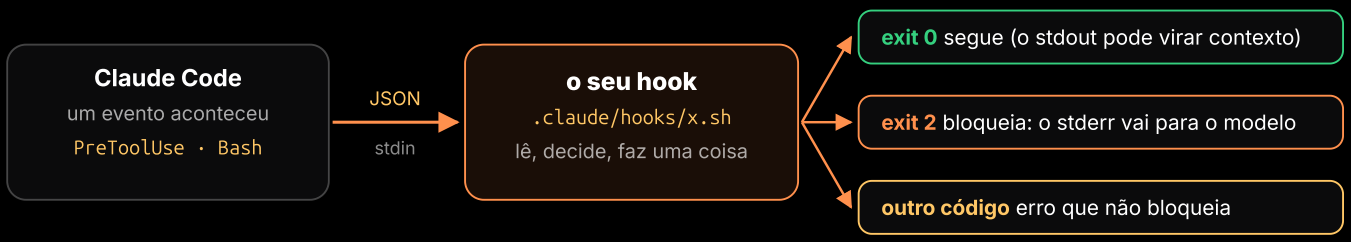
Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

A Trava é a camada determinística do harness. Ela escuta os eventos da sessão e roda hooks: não é o modelo que decide, é um programa seu que o Claude Code executa nos momentos certos, com o modelo concordando ou não.

O ponto de partida: pedir não é obrigar. Tudo que vai no CLAUDE.md, nas rules e nas skills é pedido. Entra no contexto e passa por tudo que o Módulo 1 mostrou. "Sempre rode os testes depois de editar": ele edita e roda, edita e roda, e na terceira edição pula. E não dá para depurar por que o modelo pulou. Com um hook, o teste roda depois de toda edição, porque quem roda é o Claude Code.

O contrato é um só: entra um JSON, sai um código. A aula lembra que existem outros harness, como Codex e OpenCode, e foca no Claude Code. A configuração mora no `settings.json`, que é o começo de tudo. No projeto da gravação ele estava vazio, e no seu pode nem existir ainda.



Para cada evento da sessão você pode ligar um ou mais comandos. Os seis que a aula destacou para o dia a dia. Onde o vídeo não deu um uso, a coluna marca o complemento deste material:

EVENTO	QUANDO DISPARA	PARA QUE SERVE
`SessionStart`	A sessão começa	Rodar uma dependência, avisar algo sobre o projeto
`UserPromptSubmit`	Você envia um pedido, antes do modelo ver	Conferir ou completar o pedido (complemento)
`PreToolUse`	Antes de uma ferramenta: Edit, Write, Bash, Read	Principalmente nas tools de edição e escrita, e no Bash
`PostToolUse`	Depois que a ferramenta rodou	Lint e teste depois de editar (aula 2.2)
`Notification`	O Claude espera alguma coisa de você	Avisar fora do terminal (complemento)
`Stop`	Ele terminou a resposta	Conferir antes de encerrar (complemento, aula 2.4)

Evento, matcher, comando

No `settings.json`: qual evento escutar, o matcher (qual ferramenta, por exemplo `Edit|Write`) e o comando, um `.sh` ou qualquer executável.

Libera, pergunta ou bloqueia

Libera e o modelo segue. Pergunta e você confirma. Bloqueia com `deny`, com o motivo: "comando destrutivo, é proibido". Um `migrate:fresh` não vai para a frente nunca.

> *"For static context that doesn't require a script, use CLAUDE.md instead."*
>
> Documentação oficial do Claude Code · Hooks reference · SessionStart

2 O que a aula *não teve tempo de dizer*

ALÉM DA AULA · SÓ O EXIT 2 BLOQUEIA. O EXIT 1 DEIXA PASSAR

O `exit 1` é a convenção do Unix para erro, e é o que um script quebrado devolve (jq ausente, caminho errado, typo). Para o Claude Code é "non-blocking error": aparece um aviso e a ação segue. E o `exit 2` muda de efeito conforme o evento. No `PreToolUse`, barra a ferramenta e o stderr vai para o modelo. No `UserPromptSubmit`, o pedido não chega ao modelo e o stderr aparece só para você, fora do contexto. No `PostToolUse`, a ferramenta já rodou: não desfaz nada, só mostra o stderr ao modelo. No `Stop`, o modelo não encerra e continua trabalhando. No `SessionStart`, só você vê o erro. No `Notification`, nada.

Hook de política que pode quebrar? A partir do 2.1.295 existe `"onFailure": "block"`: falha, timeout ou script ausente passam a bloquear, como um exit 2.

Fonte: Hooks reference, "Exit code output", "Exit code 2 behavior per event" e "Block the action when a hook fails". "If your hook is meant to enforce a policy, use exit 2."

ALÉM DA AULA · LIBERA NÃO É "ALLOW"

O "libera" da aula é `exit 0` sem imprimir nada: o hook não decide e o fluxo normal de permissão segue. Se ele imprimir `"permissionDecision": "allow"`, é outra coisa: pula o pedido de permissão. Um hook que devolve `allow` por engano aprova tudo o que bate no matcher. As regras de `deny` e `ask` das permissões continuam valendo, e entre hooks que discordam vence `deny`, depois `defer` (só no `-p`), depois `ask`, depois `allow`.

Teste feito para este material: o `PreToolUse` roda antes da checagem de permissão. Num `claude -p`, um comando que precisava de aprovação (e não tinha quem aprovasse) aparece no log do espião como `PreToolUse · Bash`, sem nenhum `PostToolUse` depois: ele nunca rodou.

Fonte: Hooks reference, "PreToolUse decision control".

ALÉM DA AULA · O STDOUT DE POUCOS EVENTOS VIRA CONTEXTO

Dos seis eventos da aula, só no `SessionStart` e no `UserPromptSubmit` o texto que o hook imprime com `exit 0` entra no contexto do modelo (a doc lista mais dois, `UserPromptExpansion` e `PostModelSwitch`). Nos outros vai só para o log de debug. Por isso o espião nunca imprime nada, e o `contexto-inicio.sh` imprime só fatos curtos (branch, commits, próxima tarefa), não ordens. Um `echo "debug"` esquecido num `SessionStart` vai parar na pilha de toda sessão. O teto é de 10.000 caracteres; o que passar disso vira um arquivo que o modelo não é obrigado a ler. E o `SessionStart` dispara de novo em `resume`, `/clear` e compactação: o matcher dele é a origem (`startup`, `resume`, `clear`, `compact`), não uma ferramenta.

Fonte: Hooks reference, "Exit code 0", "JSON output" e "SessionStart".

ALÉM DA AULA · O MATCHER TEM DUAS LEITURAS

Só letras, números, `_`, `-`, espaço, vírgula e `|`: nome exato ou lista exata (`Edit|Write` ou `Edit, Write`). Qualquer outro caractere: regex, sem âncora. `Edit.*` pega também o `NotebookEdit`. `mcp__github`, sem o `.*`, é comparado como nome exato e não casa com nenhuma ferramenta. Em `UserPromptSubmit` e `Stop` não existe matcher: se você puser um, é ignorado em silêncio e o hook roda sempre. No `Notification`, o matcher é o tipo do aviso, como `permission_prompt` e `idle_prompt`.

Fonte: Hooks reference, "Matcher patterns".

ALÉM DA AULA · HOOK NÃO TEM TERMINAL

O hook roda sem terminal de controle: `printf '\a' > /dev/tty` falha calado, e o bipe nunca toca. Para bipe e notificação do terminal, o hook devolve o campo `terminalSequence` e o Claude Code emite por ele. É o que o `avisa.sh` deste material faz (no WSL, é assim que o Windows Terminal mostra o aviso). E o `Notification` não é imediato: `permission_prompt` chega depois de uns 6 segundos sem você digitar, `idle_prompt` uns 60 segundos depois da resposta. No `claude -p` não há prompt de permissão nem espera ociosa, e o `terminalSequence` é ignorado: "In non-interactive mode with the `-p` flag and in the Agent SDK, it ignores the field."

Fonte: Hooks reference, "Emit terminal notifications" e "Notification".

ALÉM DA AULA · DE ONDE MAIS VEM HOOK, E QUANDO ELE RODA

Além dos três `settings.json` (usuário, projeto, local), hooks vêm de plugins (`hooks/hooks.json`), do frontmatter de skills e de subagentes, e da política da empresa. Todos se somam. O `/hooks` mostra cada um com a origem: é assim que você descobre o hook de um plugin instalado, como o bc-harness da gravação. Numa sessão interativa, nenhum hook de settings roda antes de você aceitar a confiança na pasta, nem o do seu `~/.claude`. No `claude -p` a pasta é tratada como confiável: o `.claude/settings.json` de um repositório alheio roda direto. Repositório que você não escreveu: leia o `.claude/` dele antes.

Fonte: Hooks reference, "Hook locations", "The /hooks menu", "Disable or remove hooks" e "Workspace trust". O formato deste material é o do Claude Code; em outro harness, confira a documentação dele.

3 Faça no *seu repositório*

Use o projeto em que você escolheu a feature-alvo, como na aula. O instalador coloca três hooks em `.claude/hooks/`, liga dois deles no `.claude/settings.json` e deixa o espião num arquivo à parte, que só liga quando você pede. Se você já tem um `settings.json`, os hooks são somados aos seus, com cópia do original. Nada seu é apagado.

1 Confira o jq.

Todos os hooks do módulo leem o JSON com ele.

Terminal · na raiz do seu projeto — shell

```
jq --version
```

Não tem? Instale com o gerenciador do seu sistema:

Ubuntu e WSL — shell

```
sudo apt install -y jq
```

macOS — shell

```
brew install jq
```

2 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou numa aula anterior? Só atualize:

Terminal · qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

3 Instale os arquivos da aula.

Volte para a raiz do seu projeto (`cd` no caminho dele) antes de rodar:

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 2.1
```

A saída mostra `criado` ou `somado` para cada arquivo. Um `conflito` quer dizer que você já tinha um arquivo com o mesmo nome: o da aula fica ao lado, como `.harness-lab-novo`. Ao somar, o original fica em `.claude/settings.json.antes-harness-lab`: confira e apague, ele não vai para o git. Se o seu `settings.json` tinha comentário ou vírgula sobrando, o jq não soma e o passo 4 não mostra os eventos: volte o `.antes-harness-lab`, corrija o JSON (`jq . .claude/settings.json` aponta o erro) e rode de novo.

4 Confira o que entrou.

Terminal · na raiz do seu projeto — shell

```
ls -l .claude/hooks
jq '.hooks | keys' .claude/settings.json
git check-ignore -v .claude/logs/eventos.log
```

Os três scripts com o `x` de executável, `Notification` e `SessionStart` na lista de eventos, e o `check-ignore` apontando o `.claude/logs/.gitignore`: os logs ficam fora do git.

5 Teste os hooks fora do Claude Code.

Hook é só um programa. Mande para ele o JSON que o Claude Code mandaria. Os arquivos de `modelos/entradas/` foram capturados de uma sessão real:

Terminal · na raiz do seu projeto — shell

```
A=~/harness-lab-materiais/modulo-2-trava
E=$A/aula-2.1-hooks-o-que-o-modelo-nao-decide/modelos/entradas
.claude/hooks/espiao.sh < $E/pretooluse-bash.json; echo "exit $?"
.claude/hooks/contexto-inicio.sh; echo "exit $?"
.claude/hooks/avisa.sh < $E/notification.json; echo "exit $?"
cat .claude/logs/eventos.log
```

o que aparece — no projeto de teste deste material

```
exit 0                                  # espião: nada no stdout
Branch atual: main.
Últimos commits:
  3f1c2ab inicio
tarefas.md tem 2 tarefa(s) pendente(s). Próxima: carrinho com cupom
exit 0
{"terminalSequence":"\u0007\u001b]9;Claude needs your permission..."}
exit 0
20:25:00 · PreToolUse · Bash · cat notas.txt
```

O texto do `contexto-inicio.sh` é exatamente o que o modelo vai receber no começo de cada sessão. Sem `tarefas.md` no projeto, a última linha some: crie um com linhas `- [ ] descrição`, o formato da aula 1.4, para o passo 8 funcionar. O `avisa.sh` pode abrir uma notificação do sistema já neste teste.

6 Abra o Claude Code com o espião ligado.

O `--settings` soma o arquivo do espião aos seus settings só nesta sessão. Aceite a confiança na pasta: antes disso, nenhum hook roda.

Terminal 1 · na raiz do seu projeto — claude

```
claude --settings .claude/settings.espiao.json
```

Num segundo terminal, assista ao log crescer (`Ctrl+C` para sair):

Terminal 2 · na raiz do seu projeto — shell

```
tail -f .claude/logs/eventos.log
```

7 **Confira o que está ligado e de onde veio.**

O `/hooks` lista `SessionStart` e `Notification` do settings do projeto, os oito do espião, que vieram do `--settings`, e os hooks dos seus plugins. Selecione um para ver o comando completo.

Dentro do Claude Code — sessão

```
/hooks
```

8 **Prove que o SessionStart chegou ao modelo e assista a uma tarefa.**

Dentro do Claude Code — sessão

```
> Sem ler arquivos: qual é a branch e a próxima tarefa?
> Leia o README, acrescente uma linha no fim e rode git diff
```

A primeira resposta vem do texto do `contexto-inicio.sh`, sem nenhuma ferramenta. Na segunda, o Terminal 2 mostra um par `PreToolUse` e `PostToolUse` por ferramenta. Quando ele pedir permissão para editar (no modo padrão), espere uns segundos sem digitar: o `avisa.sh` dispara e o log ganha uma linha `Notification · permission_prompt`. Saia com `/exit` para ver o `SessionEnd`.

.claude/logs/eventos.log — log real · claude -p · 2.1.316 · caminhos encurtados

```
20:24:25 · SessionStart · startup
20:24:27 · UserPromptSubmit · Sem ler nenhum arquivo, diga a branch atual...
20:24:30 · PreToolUse · Write · notas.txt
20:24:30 · PostToolUse · Write · notas.txt
20:24:30 · PreToolUse · Bash · cat notas.txt     # pediu aprovação, não rodou
20:24:32 · PreToolUse · Read · notas.txt
20:24:32 · PostToolUse · Read · notas.txt
20:24:34 · Stop
20:24:34 · SessionEnd
```

Os arquivos que o instalador colocou

.claude/settings.json — instalado pelo instalar.sh · vai para o git

```
{
  "hooks": {
    "SessionStart": [{                          // evento
      "hooks": [{ "type": "command", "timeout": 10,
        "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/contexto-inicio.sh" }]
    }],
    "Notification": [{
      "matcher": "permission_prompt|idle_prompt",  // tipo do aviso
      "hooks": [{ "type": "command", "timeout": 10,
        "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/avisa.sh" }]
    }]
  }
}
```

O caminho parte sempre de `$CLAUDE_PROJECT_DIR`, a raiz onde a sessão começou, entre aspas: o modelo dá `cd` no meio da sessão, e caminho relativo para de funcionar. O `timeout` é em segundos (sem ele, 600). A documentação atual também aceita a forma exec, sem shell: `"command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/x.sh"` com `"args": []`. O `.claude/settings.espiao.json` tem o mesmo desenho, com o `espiao.sh` nos oito eventos e `timeout` de 5.

O `avisa.sh` tenta a notificação do sistema (`notify-send` no Linux, `osascript` no macOS) e termina devolvendo um `terminalSequence` com bipe e a sequência OSC 9, que o Claude Code emite no terminal. Ghostty, Warp e urxvt usam OSC 777: o comentário no script mostra a troca. Quer o aviso em todo projeto, e não só neste? Copie o `avisa.sh` para `~/.claude/hooks/` e ligue o mesmo bloco `Notification` no `~/.claude/settings.json`, com o caminho `~/.claude/hooks/avisa.sh`.

4 Arquivos que *reforçam a aula*

`.claude/hooks/espiao.sh`

O ciclo de vida da sua sessão, evento por evento, sem interferir em nada. Ver o `PreToolUse` antes de cada ferramenta é o que torna "o modelo não decide" concreto.

No repositório: modulo-2-trava/aula-2.1-hooks-o-que-o-modelo-nao-decide/projeto/.claude/hooks/

`.claude/settings.espiao.json`

O espião ligado nos oito eventos, num arquivo que só vale quando você passa `--settings`. Fica no git sem ligar nada para o time.

No repositório: modulo-2-trava/aula-2.1-hooks-o-que-o-modelo-nao-decide/projeto/.claude/

`.claude/hooks/contexto-inicio.sh`

O "aviso sobre o projeto" do `SessionStart`: branch, últimos commits e próxima tarefa, sempre, sem depender de o modelo lembrar de ler nada.

No repositório: modulo-2-trava/aula-2.1-hooks-o-que-o-modelo-nao-decide/projeto/.claude/hooks/

`.claude/hooks/avisa.sh`

O `Notification` da aula, do jeito que funciona: `terminalSequence` no lugar do `/dev/tty`, notificação do sistema e webhook opcional (`AVISO_WEBHOOK`) para o Módulo 5.

No repositório: modulo-2-trava/aula-2.1-hooks-o-que-o-modelo-nao-decide/projeto/.claude/hooks/

`modelos/libera-pergunta-bloqueia.sh`

As três respostas do `PreToolUse` num esqueleto curto: `git status` passa, `rm -rf` pergunta, `migrate:fresh`, `prisma migrate reset` e `rails db:reset` são negados com motivo. Não é instalado: a versão completa é a da aula 2.3.

No repositório: modulo-2-trava/aula-2.1-hooks-o-que-o-modelo-nao-decide/modelos/

`modelos/entradas/*.json`

"Entra um JSON": o stdin real de seis eventos, capturado numa sessão. Serve para testar qualquer hook com `<`, sem abrir o Claude Code.

No repositório: modulo-2-trava/aula-2.1-hooks-o-que-o-modelo-nao-decide/modelos/entradas/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
O `eventos.log` não cresce	Sessão aberta sem `--settings`, confiança na pasta não aceita, script sem `chmod +x` ou jq ausente	O `/hooks` lista o espião? Se não, reabra com o passo 6. Se lista, rode o passo 5
`PreToolUse` no log sem `PostToolUse` depois	A ferramenta não rodou: a permissão negou ou um hook barrou	Normal. O `PreToolUse` roda antes da checagem de permissão
O aviso nunca aparece	Você estava digitando, ou a sessão é `claude -p`. O `Notification` espera uns 6 s (permissão) ou 60 s (ocioso)	Peça algo que precise de permissão e tire a mão do teclado
Sem bipe, sem notificação do terminal	Hook escrevendo em `/dev/tty`, que hook não tem, ou terminal que não entende OSC 9	Use `terminalSequence`, como o `avisa.sh`. Em Ghostty, Warp e urxvt, troque para OSC 777
O modelo "sabe" coisas estranhas desde o começo	`echo` de debug num hook de `SessionStart` ou `UserPromptSubmit`: o stdout virou contexto	Mande debug para arquivo ou stderr. Nesses eventos, imprima só o que o modelo deve ler
O hook "bloqueou", mas o comando rodou	Saiu com `exit 1` (ou quebrou no meio): erro que não bloqueia	`exit 2`, ou `"deny"` em JSON com `exit 0`. Para script que pode falhar, `"onFailure": "block"`
Quero uma sessão sem nenhum hook	Investigar se é um hook que está atrapalhando	Abra com o comando logo abaixo da tabela

Terminal · sessão sem nenhum hook — claude

```
claude --settings '{"disableAllHooks": true}'
```

Para ver o que cada hook fez, abra a sessão com `claude --debug`: o log fica em `~/.claude/debug/<id-da-sessão>.txt`, e as linhas com `Hooks:` trazem evento, comando, código de saída e tempo.

6 Você sai *com*

- [ ] `espiao.sh`, `contexto-inicio.sh` e `avisa.sh` executáveis em `.claude/hooks/`, e `.claude/logs/` fora do git.
- [ ] O `.claude/settings.json` com `SessionStart` e `Notification`, somado ao que você já tinha.
- [ ] Os três hooks testados com JSON no stdin, sem abrir o Claude Code.
- [ ] O `eventos.log` de uma sessão sua, do `SessionStart` ao `SessionEnd`, com um par `PreToolUse` e `PostToolUse` por ferramenta.
- [ ] O modelo respondendo a branch e a próxima tarefa sem ler nenhum arquivo.
- [ ] A diferença entre `exit 0`, `exit 2` e `exit 1`, e entre "libera" e `allow`.

PROMETIDO NA AULA

- ✓ "A carinha do `settings.json`": anotado na seção 3, pronto em `projeto/.claude/`.
- ✓ "Se você já tem um `settings.json`": o instalador soma e guarda o original (passo 3).
- ✓ Os seis eventos do dia a dia: tabela na seção 1, e o que o `exit 2` faz em cada um na seção 2.
- ✓ O `SessionStart` com "um aviso sobre o projeto": `projeto/.claude/hooks/contexto-inicio.sh`.
- ✓ "Notification, quando o Claude espera alguma coisa de você": `projeto/.claude/hooks/avisa.sh`.
- ✓ Evento, matcher e comando `.sh`: o `settings.json` anotado e o `settings.espiao.json` (seção 3).
- ✓ Libera, pergunta ou bloqueia, com o motivo do `deny`: `modelos/libera-pergunta-bloqueia.sh`.
- ✓ "E se o erro voltasse sozinho?": o hook de lint e teste é a aula 2.2 (caixa Próxima, no fim).

7 *Glossário*

Hook	Programa seu que o Claude Code roda num evento da sessão. O modelo não decide se roda.
Evento	O momento que dispara o hook: início da sessão, pedido, antes e depois de uma ferramenta, aviso, fim da resposta.
Matcher	Filtro do evento. Nas ferramentas, o nome dela (`Bash`, `Edit|Write`) ou uma regex.
stdin, stdout, stderr	O hook recebe o JSON no stdin. Dos seis eventos da aula, o stdout vira contexto só em `SessionStart` e `UserPromptSubmit`. O stderr é o motivo num `exit 2`.
permissionDecision	Resposta em JSON do `PreToolUse`: `allow`, `ask`, `deny` (e `defer`, só no `-p`).
terminalSequence	Campo de saída que pede ao Claude Code para emitir bipe ou notificação no terminal.
CLAUDE_PROJECT_DIR	Raiz onde a sessão começou. Não muda quando o modelo dá `cd`.

8 Para *ir além*

- Hooks reference: a lista dos eventos, "Exit code 2 behavior per event", "Matcher patterns" e o JSON de entrada de cada evento.
- Automate actions with hooks: exemplos prontos (notificação, formatar depois de editar) e "Limitations and troubleshooting".
- Settings: onde mora cada arquivo, a precedência e como o `--settings` entra na conta.
- Permissions: as regras de `allow`, `ask` e `deny`, que valem junto com o hook, e a confiança na pasta.

Próxima: *Aula 2.2 · O hook que roda lint e teste a cada edição*

E se o erro voltasse sozinho? Um `PostToolUse` em `Edit|Write` que roda lint e teste e devolve a falha direto para o modelo. Sem colar o erro de volta no chat.